

Advanced Topics

Advanced Topics

HTML

Web Scraping

Jupyter Notebooks

JSON

Advanced Topics Day 1

HTML

Our focus: Know enough to

- understand a page

- create a simple page

Hyper Text Markup Language

Standard language for creating Web pages

Describes the structure of a Web page

Tells web browser how to display content

Tag-based

HTML: Basic Page

```
<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>
<body>

<h1>My First Heading</h1>
<p>My first paragraph.</p>

</body>
</html>
```



HTML: Basics

Start with the
document type
declaration

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h1>My First Heading</h1>
```

```
<p>My first paragraph.</p>
```

```
</body>
```

```
</html>
```

The document

Visible part

Some Basic HTML Tags

Heading 1

← Headings (h1 to h6)

```
<h1>This is heading 1</h1>
```

```
<h2>This is heading 2</h2>
```

```
<h3>This is heading 3</h3>
```

Heading 2

Heading 3

Paragraphs

```
<p>This is a paragraph.</p>
```

```
<p>This is another paragraph.</p>
```

Heading 4

Heading 5

Links

```
<a href="https://www.w3schools.com">This is a link</a>
```

Heading 6

Images

```

```

HTML: Elements

Everything from the start tag to the end tag

`<h1>My First Heading</h1>`

`<p>My first paragraph.</p>`

start tag element content end tag

The `<html>` element is the root element

`<!DOCTYPE html>`
`<html>`
`<body>`

`<h1>My First Heading</h1>`
`<p>My first paragraph.</p>`

`</body>`
`</html>`

Elements can be nested inside other elements

The `<body>` element is nested inside the `<html>` element
The `<h1>` and `<p>` elements are nested inside the `<body>` element

Gotchas

Most element tags come in pairs - don't forget ending one! Sometimes things seem ok, but often not.

Some element tags not paired like `<br>` and contain no content (called empty elements)

Tags not case sensitive - recommend lowercase

Tag reference guide: <https://www.w3schools.com/tags/default.asp>

HTML: Attributes

All elements can have attributes

Provide additional info about elements

Specified in start tag

Usually come in name/value pairs like name="value"

Examples

- `<a href="https://www.w3schools.com">Visit W3Schools</a>`
- ``
- ``
- ``

`href, src, width, height, alt`
are some attributes

HTML: Paragraphs & Display

- Web browsers ignore extra spaces and line breaks, even in `<p>` `</p>` tags
- `<hr>` tag puts horizontal line on page
- `<br>` gives line break without starting new paragraph
- `<pre>` `</pre>` tag observes spaces and line breaks

HTML: Styles

Attribute for color, font, size, etc.

```
<tagname style="property:value;">
```

Property is a CSS property

Value is a CSS value

- Background Color
- Text Color
- Fonts
- Text Size
- Text Alignment

CSS is cool for writing pages, but not important for Data Manipulation - want to learn CSS?

https://www.w3schools.com/html/html_css.asp

See https://www.w3schools.com/html/html_styles.asp for examples.

Color info here:

https://www.w3schools.com/html/html_colors.asp

HTML: Formatting

These are paired element tags

- `<b>` - Bold text
- `<strong>` - Important text
- `<i>` - Italic text
- `<em>` - Emphasized text
- `<mark>` - Marked text
- `<small>` - Smaller text
- `<del>` - Deleted text
- `<ins>` - Inserted text
- `<sub>` - Subscript text
- `<sup>` - Superscript text

HTML: Comments

`<!--` Anything in here is commented

out. It can be multiple

lines of stuff that's commented out `-->`

HTML: Links

Click a link to go to another place either within a page or to a new page

```
<a href="https://www.w3schools.com/">Visit W3Schools.com!</a>
```



Where you will go if you
click the link



What the text of the link
says

You can make images, buttons, and other things act as links if you want:

https://www.w3schools.com/html/html_links.asp

HTML: Images

Syntax

```

```

src and alt are required (alt isn't really required, but ...)

Note: one of the few unpaired tags

Examples

```

```

```

```

Path for src

Absolute vs Relative paths

HTML: Tables

Default is no border (hard to read)

Player	Goals	Team
Erin Cuthbert	5	Chelsea
Katie McCabe	3	Arsenal

Player	Goals	Team
Erin Cuthbert	5	Chelsea
Katie McCabe	3	Arsenal

HTML: Table Anatomy

Default is no border (hard to read)

Player	Goals	Team
Erin Cuthbert	5	Chelsea
Katie McCabe	3	Arsenal

Player	Goals	Team
Erin Cuthbert	5	Chelsea
Katie McCabe	3	Arsenal

Tables consist of table rows `<tr>` `</tr>`

To get the bolded look for the column names, inside the `<tr>` `</tr>`, we have *table header* tags `<th>` `</th>`

To get the non-bolded look for data, we use *table data* `<td>` `</td>` tags

To get the border, we add a border attribute to the `<table>` `</table>` tags

HTML: Tables

Default is no border (hard to read)

Player	Goals	Team
Erin Cuthbert	5	Chelsea
Katie McCabe	3	Arsenal

Replace `<table>` with
`<table border=1>` to
go from
this to
this

Player	Goals	Team
Erin Cuthbert	5	Chelsea
Katie McCabe	3	Arsenal

```
<!DOCTYPE html>
<html>
<body>

<h2>A basic HTML table</h2>

<table>
  <tr>
    <th>Player</th>
    <th>Goals</th>
    <th>Team</th>
  </tr>
  <tr>
    <td>Erin Cuthbert</td>
    <td>5</td>
    <td>Chelsea</td>
  </tr>
  <tr>
    <td>Katie McCabe</td>
    <td>3</td>
    <td>Arsenal</td>
  </tr>
</table>

</body>
</html>
```

HTML: Tables

Lots more styling for tables: https://www.w3schools.com/html/html_tables.asp

HTML: Page Title

Shown in browser's title bar

```
<!DOCTYPE html>
<html>
<head>
  <title>HTML Tutorial</title>
</head>
<body>
```

The content of the document.....

```
</body>
</html>
```



HTML: Lists

Bulleted lists

`<ul>`

`<li>Dragons</li>`

`<li>Possums</li>`

`<li>Dogs</li>`

`<li>Anteaters</li>`

`</ul>`

- Dragons
- Possums
- Dogs
- Anteaters

Numbered Lists (same as bulleted lists but use `<ol>` and `</ol>` instead of `<ul>` and `</ul>`)

HTML Lab

Lab linked to Moodle

Advanced Topics Day 2

Web Scraping

Automated data collection from web sites

Grab HTML data & convert to csv

Preferable to doing it by hand, especially with large data sets or data that changes

Web Scraping Tools

Can build your own

Can use pre-built one

Beautiful Soup: <https://pypi.org/project/beautifulsoup4/>

Others, too

Web Scraping

Building part of a simple scraping tool: See activity linked on Moodle - when you finish the handout, come back to these slides

Web Scraping

The last activity got painful at the end, and that's an **easy** one compared to others!

What if we need (more commonly) to get data out of tables?

Web Scrapping

We can build a DataFrame from a list of column headers and a 2D list of data like this:

```
1 import pandas as pd
2
3 l_band_info = [['Kiss', 'Forever', 1990], \
4                ['Danger Danger', 'Rock America', 1992], \
5                ['Heart', 'Alone', 1985]]
6
7 l_column_names = ['Band', 'Song', 'Year']
8 df = pd.DataFrame(l_band_info, columns = l_column_names)
9 print(df)
10
```

Web Scraping

Assume you have a single web page that contains a single table.

You don't know where the table is in the page or how many rows and columns it has.

Try to build a simple program that scrapes every item from the table into

- a list of the things in the `<th>` `</th>` tags (for the column headers)
- a 2D list of the things in the `<td>` `</td>` tags (for the data)

so we can build a DataFrame out of them.

When you do the scraping, assume that the `<th>`, `</th>`, `<td>`, and `</td>` tags look exactly like that and don't contain any spaces or attributes in them.

Your program should work for different size tables (different numbers of rows and columns).

End Of Day 2

Stop here!

Advanced Topics Day 3

We've now built tiny “scrapers” that work under precise conditions.

Time to build ones that work under more realistic conditions.

Enter Beautiful Soup...

Beautiful Soup

<https://www.crummy.com/software/BeautifulSoup/bs4/doc/>

```
pip install beautifulsoup4
```

Web Scraping

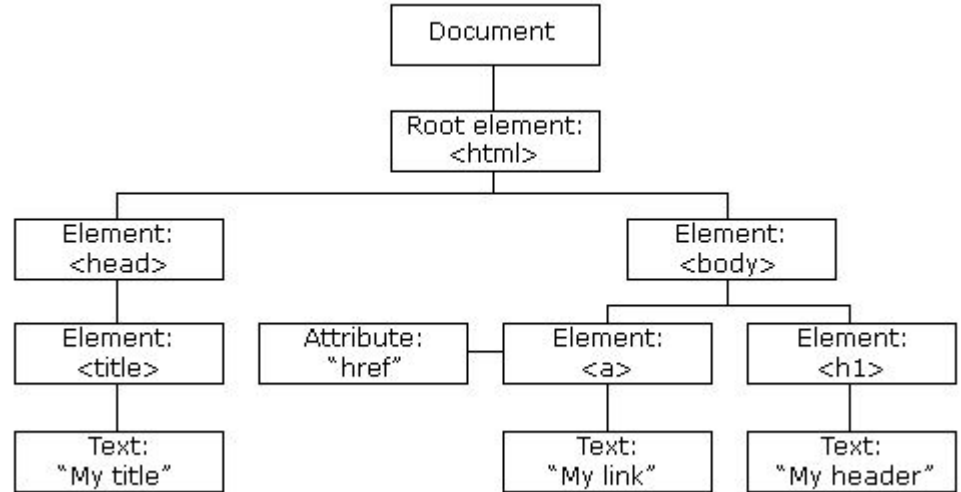
Remember that HTML pages have a structure.

They're composed of elements with a root element.

That will help us!

Let's Look At A Sample

```
<html>  
  <head>  
    <title>My title</title>  
  </head>  
  <body>  
    <a href=http://www.yahoo.com>My link</a>  
    <h1>My header</h1>  
  </body>  
</html>
```



Steps To Scrape

1. Imports

```
from bs4 import BeautifulSoup
```

2. Call the BeautifulSoup constructor

with open("index.html") as fp:

```
soup = BeautifulSoup(fp, 'html.parser')
```

<https://www.freecodecamp.org/news/with-open-in-python-with-statement-syntax-example/>



3. Use it! You get a tree of objects back. Our focus is on tag objects. Tag objects have a set of methods and attributes (just some here...)

- `print(some_tag.name)`
- `print(some_tag.attrs)`
- `print(some_tag.getText())`
- `print(some_tag.get('some attribute'))`

Navigating The Tree

Options:

- Say the name of the tag you want explicitly - want the head tag? Say `soup.head`
 - Can say `soup.body.b` to get the first `<b>` tag beneath the `<body>` tag.
- Navigate by going level by level using the list of children of a node - a tag has a **list** of children called `contents`
 - Can say

```
head_tag = soup.head
```

```
heads_children = head_tag.contents
```

```
for kid in heads_children:
```

```
    print(kid.name)
```

Navigating The Tree

There's so much more you can do for navigation

Commands to go up, sideways, back and forth

Getting What You Need

Assume we started with this:

```
from bs4 import BeautifulSoup  
  
with open("index.html") as fp:  
    soup = BeautifulSoup(fp, 'html.parser')
```

Need to find the first node?

```
first_img_node = soup.find('img')
```

Need to find all nodes?

```
l_img_nodes = soup.find_all('img')
```

Need to find the actual link?

```
actual_link = first_img_node.get('src')
```

Example

Saw lots of challenges in getting all img links if html was not written just so

For this file, our program only finds <http://pictures.com/smiley.png>

```
file_J.html x
1 <html>
2   <body>
3     
4     <img sRc="http://pictures.com/hound.png">
5     <img SRC="http://pictures.com/ooze.png">
6     
7     
8   </body>
9 </html>
10
```

Example

soup = BeautifulSoup(fd, 'html.parser')

With BeautifulSoup:

```
1 from bs4 import BeautifulSoup
2
3
4 def main():
5     d_image_links = {}
6
7     with open("file J.html", "r") as fd:
8         soup = BeautifulSoup(fd, features="lxml")
9         root = soup.html
10        img_links = root.find_all('img')
11        for item in img_links:
12            if item in d_image_links:
13                d_image_links[item.get('src')] = d_image_links[item.get('src')]
14            else:
15                d_image_links[item.get('src')] = 1
16
17        for key in d_image_links.keys():
18            print(key, d_image_links[key])
19
20    print("done")
21
22 main()
```

if item.get('src') in d_image_links:

```
toth@toth-Latitude-5520 ~/Desktop/soup
$ python3 mod_solution_scraper.py
http://pictures.com/troll.png 1
http://pictures.com/hound.png 1
http://pictures.com/ooze.png 1
http://pictures.com/smiley.png 1
http://pictures.com/ball.png 1
done
```

Tasks

1. Download the `scrape_me.html` page from Moodle.
2. Write a Python program with BeautifulSoup that prints out all of the links in `<a>` tags on the page. You do NOT need to count the frequency like we did with the images.
3. Write a Python program with BeautifulSoup that prints out all of the text in `<h3>` tags on the page.
4. Write a Python program with BeautifulSoup that finds all tables on the page and turns them into DataFrames. **Hint:** This is challenging, so you want to start small and build up to it. Make a copy of the html file that only has 1 table. Figure out how to get it working for that. Now you can try to make it work for pages with multiple tables. The final program is only about 40 lines of code.

Advanced Topics Day 4

Go Over Scrapping A Table

“Failing to plan is planning to fail.” Alan Lakein

So let's plan!

1. What is our destination/goal?

Turn tables on HTML page into DataFrames

2. Now how do we get there?

Find a table

Turn it into a 1D list of column headers and a 2D list of data

Repeat for all tables

Go Over Scrapping A Table

Start w/ a `<table>` element

Draw table on board & show decomposition!

New Lab

Tasks:

- Building an HTML page from Excel
 - Download the grades.xlsx file from Moodle.
 - Read it into a DataFrame with Pandas.
 - Use Python to write the DataFrame to an HTML page with the data in a table.
- Rip apart lists to form a DataFrame
 - Download the new version of scrape_me.html from Moodle.
 - You're told that page has a number of sections. Some of the sections contain unordered lists. Those sections contain a element with a title of the section.
 - Your job is to grab the data from the sections that contain the unordered lists and turn it into a single table, with the column headers being the text in the elements and the data for the column being the contents of the list.
- Challenge: Think about how we could get the data from the wsl page:
<https://fbref.com/en/comps/189/stats/Womens-Super-League-Stats>

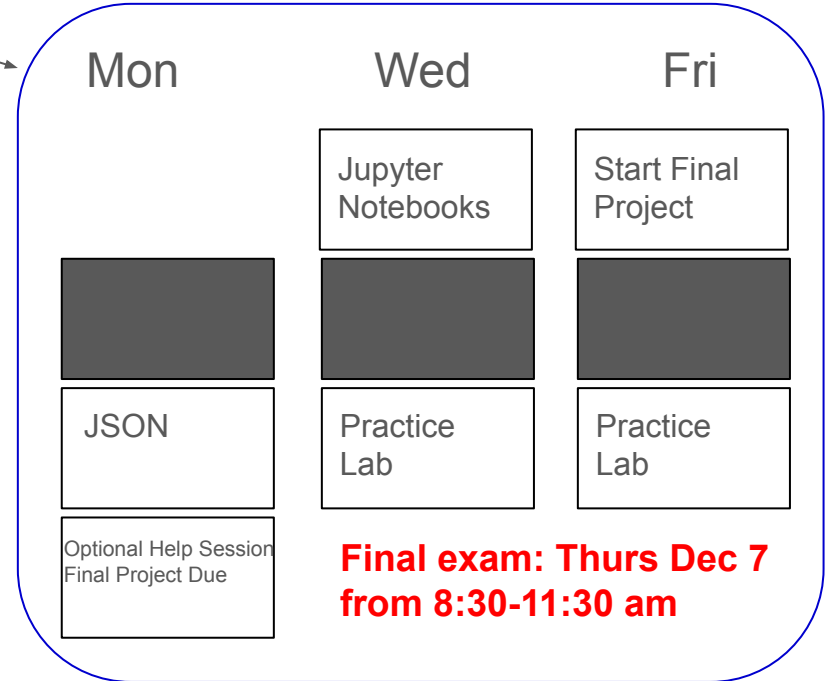
Friday Nov 10

Project work

Building our cheat sheet for the exam

Wed Nov 15

- Rest of semester plan
- Final project
 - Topics
 - Jupyter Notebooks (today's topic)
 - Web scraping
 - 2D lists
 - Dictionaries
- Final exam
 - Cheat sheet coming Monday Nov 27
 - Topics
 - Same as project, plus...
 - Essay about what you learned (can be done sooner, handed out today on Moodle)



Development Environment For Data Scientists

Most data scientists use a nicely configured environment.

We haven't been due to several reasons, including IT challenges

Typically, going to install anaconda <https://www.anaconda.com/>, getting all sorts of packages so we don't need to install everything manually.

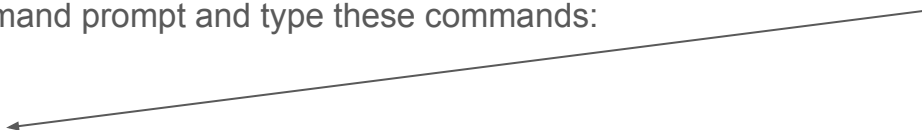
Should include Jupyter Notebooks

Install & Run Jupyter Notebooks

Installation: `pip3 install jupyter`

Running: Open a command prompt and type these commands:

Replace my username
with yours!



```
c:
cd Users\david.toth\AppData\Roaming\Python\Python311\Scripts
jupyter.exe notebook
```

If it opens in a browser, great! If not, will likely open up notepad++ because our system isn't configured well. In that case, find the line like:

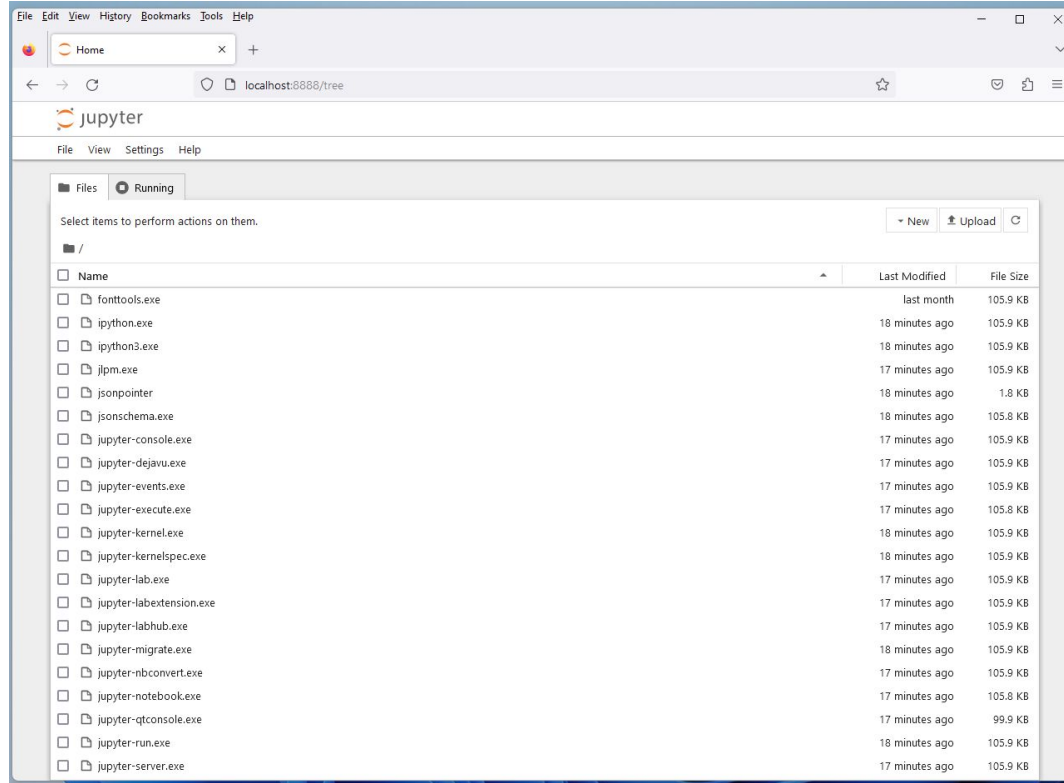
This page should redirect you to a Jupyter application. If it doesn't, `<a href="http://localhost:8888/tree?token=16b51ef2cf33e523b31b9177a7a17266f047f3f38d6fba4e">click here to go to Jupyter</a>`.

Copy the link and paste it into a web browser. Wait a few seconds...

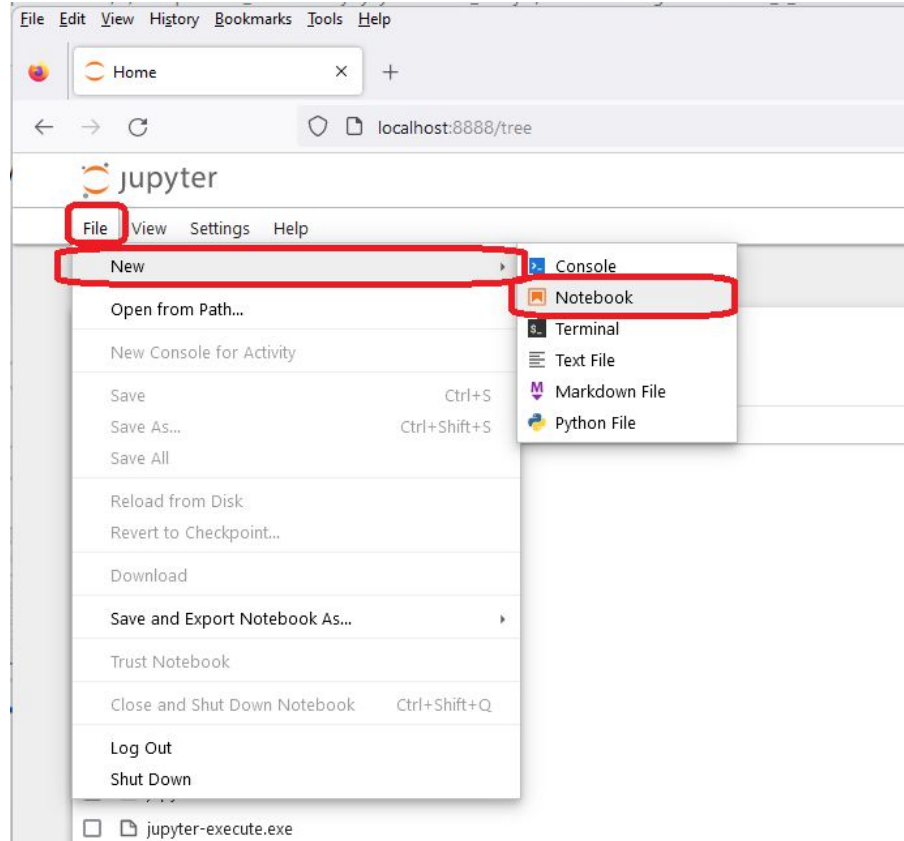


Can fix by changing file
association for html files

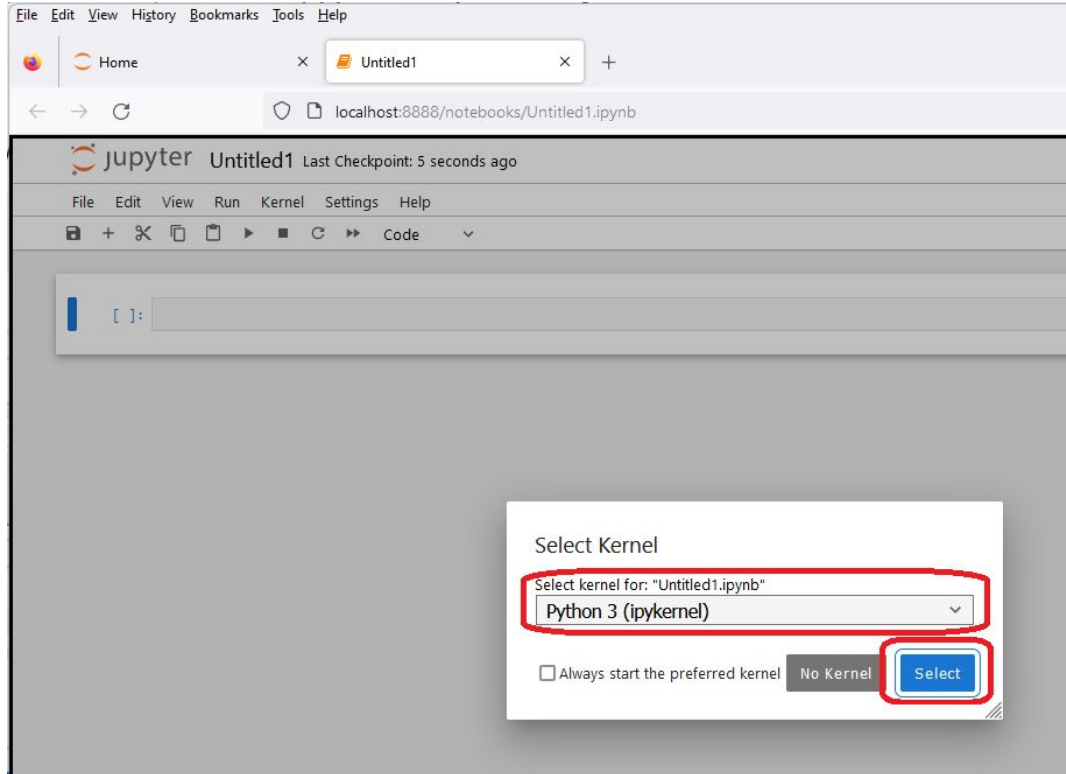
Jupyter Notebooks



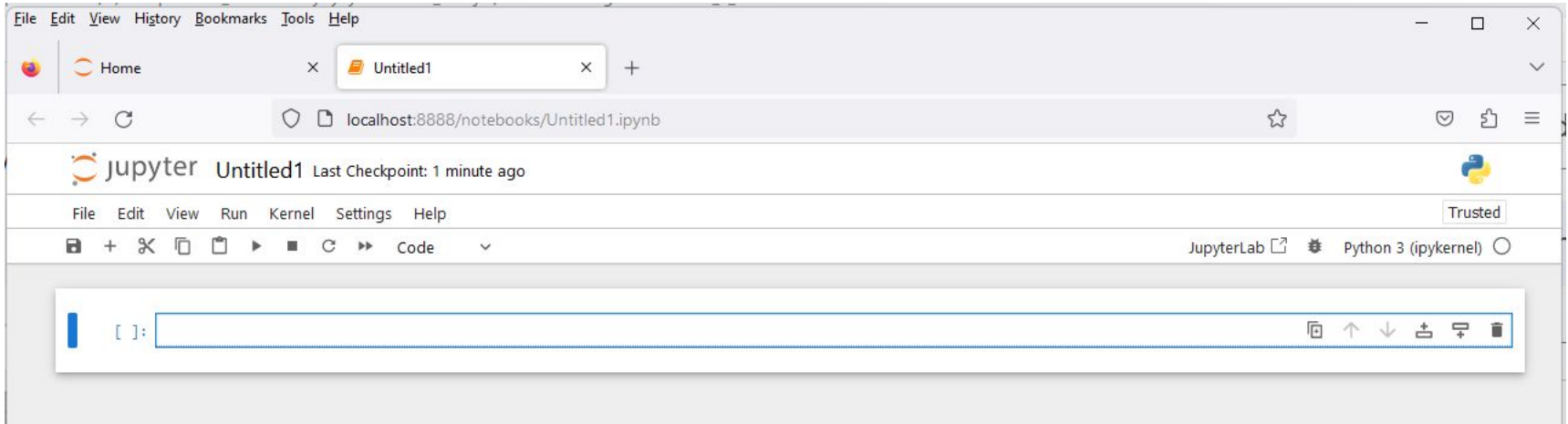
Jupyter Notebooks



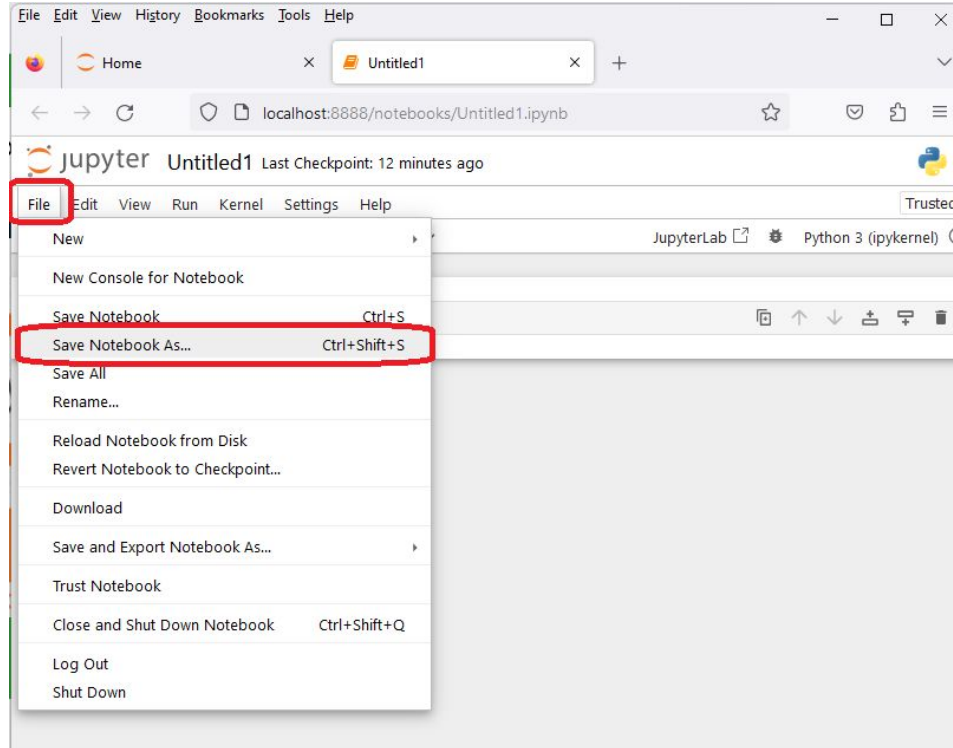
Jupyter Notebooks



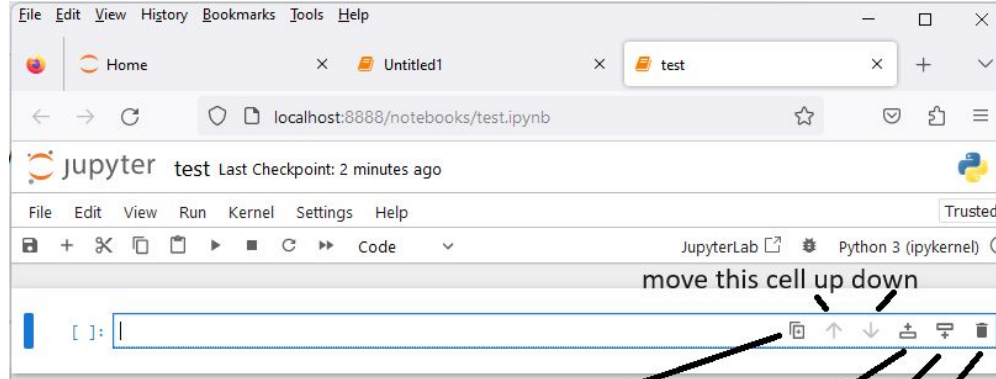
Jupyter Notebooks



Jupyter Notebooks



Jupyter Notebooks



make a duplicate
of this cell below

insert a cell above

insert a cell below

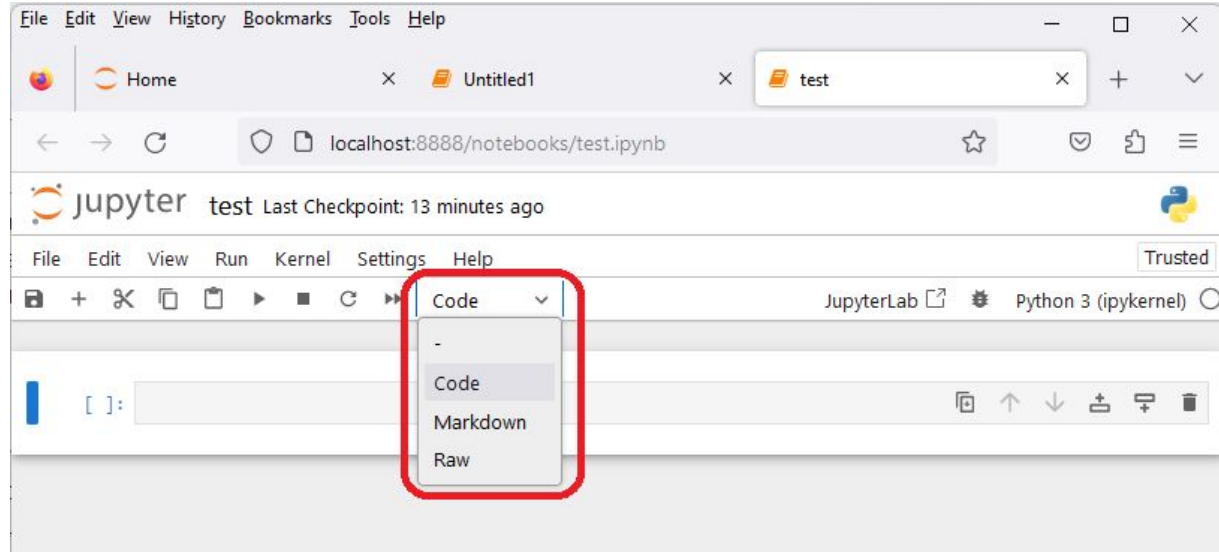
delete this cell

Jupyter Notebooks

3 kinds of cells

1. Code
2. Markdown
(formatted text)
3. Raw (don't worry for now)

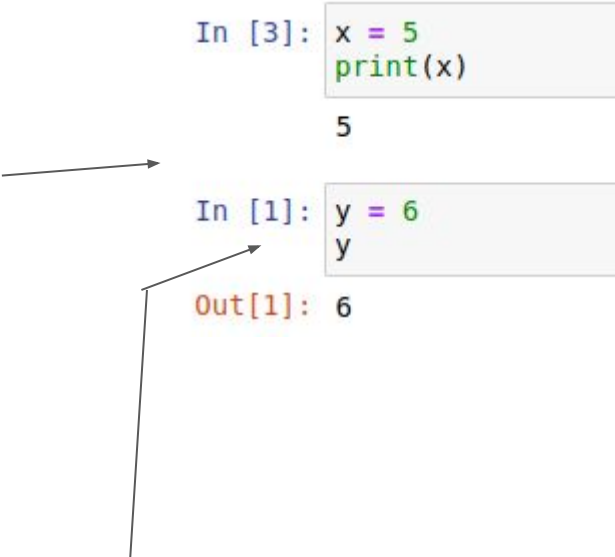
Can double-click cell to edit



Jupyter Notebooks: Code

- Put code in a cell
- Click  to execute it (or press shift & enter)
- Note two different ways to view variable's value

- A program can be split into multiple cells - careful to make sure execution of things happens in order!
- Cell menu -> Clear output, Run All Above, Run All
- Kernel menu: Restart & Clear Output, Restart & Run All



```
In [3]: x = 5  
print(x)  
  
5
```

```
In [1]: y = 6  
y  
  
Out[1]: 6
```

Num in braces indicates
order cells were run in

Jupyter Notebooks: Markdown

- # top-level header
- Add more # for smaller headers
- **bold**
- *italics*
- `code formatted`, ````` gets different format`````
- Bulleted list: start consecutive lines with -
- Numbered list: start lines with numbers followed by a period
- Hyperlinks: [text to display] (url)

Note that's a backtick, not an apostrophe (use ~ key)



Jupyter Notebooks

Dave - Pull up Jupyter Notebooks and do a quick demo!

Jupyter Notebooks

Lab activity: linked to Moodle

Friday Nov 17: Final Project

Monday Nov 27

- Cheat sheet for final exam:
https://docs.google.com/document/d/1_f9mqj5CD2l3ONZlto5FkL_oBIXK6HUAIEo6NEcd6bU/edit?usp=sharing
- JSON
- Using the command line
- Practice lab
- Work on project
- Work on essay

JSON

JavaScript Object Notation

Text format for storing data

Sample JSON string: **'{"name":"John", "age":30, "car":null}'**

Defines an object with 3 properties, each with a value

- name
- age
- car

JSON

Sample JSON string: `'{"name":"John", "age":30, "car":null}'`

Format:

- Data is in name/value pairs
- Data is separated by commas
- Curly braces hold objects
- Square brackets hold arrays
- keys must be strings, written with double quotes

JSON

Example that defines an employees object with an array of 3 employees

```
{ "employees": [  
  { "firstName": "John", "lastName": "Doe" },  
  { "firstName": "Anna", "lastName": "Smith" },  
  { "firstName": "Peter", "lastName": "Jones" }  
]}
```

Want more? Go to https://www.w3schools.com/js/js_json_intro.asp

Using the Command Line

- Windows, Mac, Linux: similarities and differences
- Command completion (tab)
- Slashes
- Blah blah blah... - talk about lots of stuff here!

Practice Lab

Linked to Moodle

- Download the two files in today's Moodle section
- Upload them to Jupyter Notebooks's workspace (see step 10 on the Jupyter Notebooks lab if you forgot how)

https://docs.google.com/document/d/12iBjG8gX4QoCBS9MtsC2_c9dIRcqJRbQqyxgQJ6LMPg/edit?usp=sharing

Work on Project

Work on Essay